

Modul 2: Backpropagation dan Automatic Differentiation

Cell 1 - markdown

Modul 2: Backpropagation dan Automatic Differentiation

****Nama:**** Fadil Alfarizzi
****NIM:**** 123450048
****Kelas:**** RC
****Tanggal:**** 2026-09-26

Berkas ini dikerjakan berdasarkan panduan Modul 2.

Cell 2 - markdown

Petunjuk

Notebook ini memuat jawaban pre-lab, turunan manual, pembandingan dengan PyTorch autograd, gradient checking, diagnosis training loop, serta tugas individu Modul 2.

Luaran: `M02\_123450048.ipynb`, `M02\_123450048.pdf`, dan `M02\_123450048\_metrics.csv`.

Cell 3 - code

```
import platform
import random

import numpy as np
import pandas as pd
import torch
from torch import nn

NIM = '123450048'
SEED = int(str(NIM)[-4:])
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

seed_everything(SEED)
torch.set_default_dtype(torch.float64)
pd.set_option('display.precision', 8)
print({'python': platform.python_version(), 'numpy': np.__version__,
      'torch': torch.__version__, 'device': str(DEVICE), 'seed': SEED})

{'python': '3.12.14', 'numpy': '2.3.5', 'torch': '2.8.0+cpu', 'device': 'cpu', 'seed': 48}
```

Cell 4 - markdown

A. Pre-lab - 10 poin

- **Gradien lokal vs gradien total pada satu simpul:**** gradien lokal adalah turunan keluaran simpul terhadap masukannya sendiri. Gradien total dari loss terhadap simpul diperoleh dengan aturan rantai, yaitu gradien dari bagian sesudah simpul dikalikan gradien lokal simpul.
- **Mengapa `backward()` hanya dapat dipanggil pada tensor skalar:**** backward membutuhkan satu nilai gradien awal, yaitu $dL/dL=1$. Untuk tensor non-skalar harus diberikan vektor gradien eksternal yang ukurannya sama.
- **Isi `.grad` bila `backward()` dipanggil dua kali tanpa `zero\_grad()`**** PyTorch menjumlahkan gradien baru ke `.grad` yang lama, sehingga nilainya menjadi dua kali untuk graf dan input yang sama.
- **Mengapa turunan BCE-with-logits terhadap logit berbentuk $\$p-y\$$, bukan $\$-y/p\$$ **** BCE-with-logits mencakup target positif dan negatif serta sigmoid secara stabil. Setelah aturan rantai diterapkan, turunan loss terhadap logit menyederhana menjadi $\$p-y\$$.

****Graf komputasi Kasus 1:****

$\$ \mathbf{x} \rightarrow z^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \rightarrow \mathbf{h} = \text{ReLU}(z^{(1)}) \rightarrow z^{(2)} = \mathbf{W}^{(2)} \mathbf{h} + \mathbf{b}^{(2)} \rightarrow p = \text{sigma}(z^{(2)}) \rightarrow \mathcal{L} = \text{BCE}(p, y) \$$

Gradien lokalnya adalah $\$ \partial z^{(1)} / \partial \mathbf{W}^{(1)} = \mathbf{W}^{(1)} \$$, $\$ \partial h_i / \partial z_i = 1 [z_i > 0] \$$, $\$ \partial z^{(2)} / \partial \mathbf{h} = \mathbf{W}^{(2)} \$$, dan $\$ \partial \mathcal{L} / \partial z^{(2)} = p - y \$$.

B. Turunan manual - 20 poin

Kasus 1 memakai nilai tetap berikut. Dengan $z^{(1)} = [1.5, 1.0]^T$, $h = [1.5, 1.0]^T$, $z^{(2)} = 2.5$, dan $p = \sigma(2.5) = 0.92414182$:

1. $\frac{\partial \text{mathcal{L}}}{\partial z^{(2)}} = p - y = -0.07585818$.
2. $\frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{W}}^{(2)}} = (p - y) \text{mathbf{h}} = [-0.11378727, -0.07585818]^T$.
3. $\frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{b}}^{(2)}} = p - y = -0.07585818$.
4. $\frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{h}}} = (p - y) \text{mathbf{W}}^{(2)} = [-0.15171636, 0.07585818]^T$.
5. $\frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{z}}^{(1)}} = \frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{h}}} \odot 1_{[z^{(1)} > 0]} = [-0.15171636, 0.07585818]^T$.
6. $\frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{W}}^{(1)}} = \left(\frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{z}}^{(1)}} \right) \text{mathbf{x}}^T = [-0.30343272, 0.15171636, -0.15171636, 0.07585818]^T$.
7. $\frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{b}}^{(1)}} = \frac{\partial \text{mathcal{L}}}{\partial \text{mathbf{z}}^{(1)}}$.

Shape gradien: $\text{dW1} = (2,2)$, $\text{db1} = (2,)$, $\text{dW2} = (2,)$, dan db2 skalar.

Cell 6 - code

```
x = np.array([2.0, -1.0])
w1 = np.array([[0.5, -0.5], [1.0, 1.0]])
b1 = np.array([0.0, 0.0])
w2 = np.array([2.0, -1.0])
b2 = 0.5
y = 1.0

def forward(x, w1, b1, w2, b2, y):
    z1 = w1 @ x + b1
    h = np.maximum(z1, 0.0)
    z2 = w2 @ h + b2
    p = 1.0 / (1.0 + np.exp(-z2))
    loss = np.logaddexp(0.0, z2) - y * z2
    return {'z1': z1, 'h': h, 'z2': z2, 'p': p, 'loss': loss}

nilai = forward(x, w1, b1, w2, b2, y)
print({k: np.round(v, 6) for k, v in nilai.items()})
assert np.allclose(nilai['z1'], [1.5, 1.0])
assert np.isclose(nilai['z2'], 2.5)
assert np.isclose(nilai['loss'], 0.0788897, atol=1e-6)
print('forward pass sesuai Kasus 1')
```

```
{'z1': array([1.5, 1. ]), 'h': array([1.5, 1. ]), 'z2': np.float64(2.5), 'p': np.float64(0.924142), 'loss': np.float64(0.07889)}
forward pass sesuai Kasus 1
```

Cell 7 - code

```
def backward(x, w1, w2, y, nilai):
    dz2 = nilai['p'] - y
    dw2 = dz2 * nilai['h']
    db2 = dz2
    dh = dz2 * w2
    dz1 = dh * (nilai['z1'] > 0.0)
    dw1 = np.outer(dz1, x)
    db1 = dz1
    return {'w1': dw1, 'b1': db1, 'w2': dw2, 'b2': db2}

grad_manual = backward(x, w1, w2, y, nilai)
for nama, v in grad_manual.items():
    print(f'{nama:>3}: {np.round(v, 7)}')
assert np.allclose(grad_manual['w2'], [-0.1137873, -0.0758582], atol=1e-6)
assert grad_manual['w1'].shape == w1.shape
print('gradien manual sesuai angka acuan')
```

```
W1: [[-0.3034327 0.1517164]
 [ 0.1517164 -0.0758582]]
b1: [-0.1517164 0.0758582]
W2: [-0.1137873 -0.0758582]
b2: -0.0758582
gradien manual sesuai angka acuan
```

Cell 8 - markdown

C. Autograd - 20 poin

Bangun ulang Kasus 1 dengan tensor PyTorch, lalu bandingkan gradiennya dengan hasil bagian B.

Cell 9 - code

```
tw1 = torch.tensor(w1, requires_grad=True)
tb1 = torch.tensor(b1, requires_grad=True)
tw2 = torch.tensor(w2, requires_grad=True)
tb2 = torch.tensor(b2, requires_grad=True)
tx, ty = torch.tensor(x), torch.tensor(y)
kriteria = nn.BCEWithLogitsLoss()
```

```
def forward_torch():
    z1 = tw1 @ tx + tb1
```

```

h = torch.relu(z1)
z2 = tw2 @ h + tb2
return kriteria(z2, ty)

loss = forward_torch()
loss.backward(retain_graph=True)
for nama, t in [('W1', tw1), ('b1', tb1), ('W2', tw2), ('b2', tb2)]:
    selisih = np.max(np.abs(t.grad.numpy() - grad_manual[nama]))
    print(f'{nama}: autograd = {np.round(t.grad.numpy(), 7)}    selisih maks = {selisih:.2e}')
    assert selisih < 1e-10
print('autograd cocok dengan backward manual')

```

W1: autograd = [[-0.3034327 0.1517164]
[0.1517164 -0.0758582]] selisih maks = 0.00e+00
b1: autograd = [-0.1517164 0.0758582] selisih maks = 0.00e+00
W2: autograd = [-0.1137873 -0.0758582] selisih maks = 0.00e+00
b2: autograd = -0.0758582 selisih maks = 0.00e+00
autograd cocok dengan backward manual

Cell 10 - code

```

grad_sebelum = tw2.grad.detach().clone()
loss.backward()
grad_sesudah = tw2.grad.detach().clone()
print('gradien W2 sebelum backward kedua:', grad_sebelum.numpy())
print('gradien W2 sesudah backward kedua:', grad_sesudah.numpy())
assert torch.allclose(grad_sesudah, 2.0 * grad_sebelum)
for tensor in (tw1, tb1, tw2, tb2):
    tensor.grad = None
loss = forward_torch()
loss.backward()
print('gradien dihitung ulang setelah gradien dikosongkan:', tw2.grad.numpy())

```

gradien W2 sebelum backward kedua: [-0.11378727 -0.07585818]
gradien W2 sesudah backward kedua: [-0.22757454 -0.15171636]
gradien dihitung ulang setelah gradien dikosongkan: [-0.11378727 -0.07585818]

Cell 11 - markdown

****Penjelasan akumulasi gradien:**** `backward()` kedua menambahkan gradien ke `.grad`, sehingga pada kasus ini nilainya menjadi dua kali gradien satu lintasan. Karena itu training loop perlu memanggil `optimizer.zero\_grad()` sebelum backward pada setiap batch agar gradien antar-langkah tidak tercampur.

Cell 12 - markdown

D. Gradient checking - 20 poin

Bandingkan gradien analitik dengan selisih terpusat:

$$g_{\text{num}} = \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon},$$

$$\text{rel err} = \frac{|g_{\text{analitik}} - g_{\text{num}}|}{|g_{\text{analitik}}| + |g_{\text{num}}| + 10^{-12}}$$

Ambang lulus: seluruh baris di bawah 10^{-5} .

Cell 13 - code

```

EPS = 1e-5

def loss_dengan(param, i, delta):
    params = {'W1': w1.copy(), 'b1': b1.copy(), 'W2': w2.copy(), 'b2': float(b2)}
    if i == ():
        params[param] += delta
    else:
        params[param][i] += delta
    return forward(x, params['W1'], params['b1'], params['W2'], params['b2'], y)['loss']

def finite_difference(param, i):
    return (loss_dengan(param, i, EPS) - loss_dengan(param, i, -EPS)) / (2.0 * EPS)

indeks = [(('W1', (0, 0)), ('W1', (0, 1)), ('W1', (1, 0)), ('W1', (1, 1)))
          + (('b1', (0,)), ('b1', (1,)))
          + (('W2', (0,)), ('W2', (1,)))
          + (('b2', ()),)]

baris = []
for nama, i in indeks:
    manual = grad_manual[nama][i] if i != () else grad_manual[nama]
    auto = {'W1': tw1, 'b1': tb1, 'W2': tw2, 'b2': tb2}[nama].grad.numpy()
    auto = auto[i] if i != () else auto
    numerik = finite_difference(nama, i)
    rel = abs(manual - numerik) / (abs(manual) + abs(numerik) + 1e-12)
    baris.append({'parameter': nama, 'indeks': str(i), 'manual': manual,
                  'autograd': float(auto), 'numerik': numerik, 'rel_err': rel})

tabel = pd.DataFrame(baris)
print(tabel.to_string(index=False))
print('relative error maksimum:', tabel['rel_err'].max())
assert len(tabel) == 9
assert tabel['rel_err'].max() < 1e-5

```

parameter indeks manual autograd numerik rel_err
W1 (0, 0) -0.30343272 -0.30343272 -0.30343272 1.07313401e-10
W1 (0, 1) 0.15171636 0.15171636 0.15171636 3.41358596e-11
W1 (1, 0) 0.15171636 0.15171636 0.15171636 3.41358596e-11
W1 (1, 1) -0.07585818 -0.07585818 -0.07585818 1.12219224e-10
b1 (0,) -0.15171636 -0.15171636 -0.15171636 3.41358596e-11
b1 (1,) 0.07585818 0.07585818 0.07585818 1.12219224e-10
W2 (0,) -0.11378727 -0.11378727 -0.11378727 8.29208263e-11
W2 (1,) -0.07585818 -0.07585818 -0.07585818 1.12219224e-10
b2 () -0.07585818 -0.07585818 -0.07585818 1.12219224e-10
relative error maksimum: 1.1221922358447185e-10

Cell 14 - code

```
tabel.insert(0, 'run_id', 'gradcheck')
tabel.insert(1, 'module', 'M02')
tabel.insert(2, 'student_id', NIM)
tabel.insert(3, 'seed', SEED)
tabel.to_csv(f'M02_{NIM}_metrics.csv', index=False)
print('tersimpan:', f'M02_{NIM}_metrics.csv')
```

tersimpan: M02_123450048_metrics.csv

Cell 15 - markdown

****Checkpoint menit ke-95.**** Tunjukkan tabel sembilan baris di atas kepada asisten sebelum melanjutkan ke bagian E.

Cell 16 - markdown

E. Diagnosis training loop - 20 poin

Fungsi `train_rusak` di bawah berjalan ****tanpa pesan galat****, tetapi memuat ****empat**** kesalahan. Kasus yang dipakai adalah XOR dengan protokol modul: FNN $\rightarrow$ 4 $\rightarrow$ 1\$, SGD $\eta=0.1$, 400 epoch, satu batch penuh.

Cell 17 - code

```
X_xor = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_xor = torch.tensor([[0.0], [1.0], [1.0], [0.0]])

def train_rusak(epoch: int = 400, lr: float = 0.1):
    """Versi bermasalah yang tetap dapat direproduksi pada torch versi kini.

    Kesalahan urutan step/backward dicatat pada diagnosis. Di sini backward
    dilakukan sebelum step agar autograd tidak menolak inplace update; tiga
    kesalahan lain tetap sengaja dipertahankan.
    """
    seed_everything(SEED)
    model = nn.Sequential(nn.Linear(2, 4), nn.Linear(4, 1))
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []
    for _ in range(epoch):
        logits = model(X_xor)
        loss = kriteria(torch.sigmoid(logits), y_xor)
        loss.backward()
        opt.step()
        riwayat.append(loss.item())
    return model, riwayat

model_rusak, riwayat_rusak = train_rusak()
print(f'loss awal : {riwayat_rusak[0]:.4f}')
print(f'loss akhir : {riwayat_rusak[-1]:.4f}')
with torch.no_grad():
    print('prediksi :', (torch.sigmoid(model_rusak(X_xor)) > 0.5).int().flatten().tolist())
    print('target :', y_xor.int().flatten().tolist())
```

loss awal : 0.7368

loss akhir : 0.6931

prediksi : [0, 0, 0, 0]

target : [0, 1, 1, 0]

Cell 18 - markdown

Temuan kesalahan

| No | Baris kode bermasalah | Mengapa keliru | Gejala yang terlihat |

|---|-----|-----|-----|

| 1 | Dua `Linear` langsung berurutan | Tanpa aktivasi nonlinier, komposisi dua layer tetap fungsi linear sehingga tidak dapat memodelkan XOR. | Loss sulit turun dan prediksi tidak memisahkan XOR. |

| 2 | `kriteria(torch.sigmoid(logits), y_xor)` | `BCEWithLogitsLoss` sudah mengandung sigmoid; sigmoid ganda membuat gradien salah dan tidak stabil. | Optimisasi lambat atau berhenti pada loss yang kurang baik. |

| 3 | `opt.step()` sebelum `loss.backward()` | Parameter diperbarui sebelum gradien baru dihitung, sehingga langkah pertama memakai gradien kosong/lama. | Update awal tidak efektif dan hasil training tidak konsisten. |

| 4 | Tidak ada `opt.zero_grad()` | Gradien PyTorch bersifat akumulatif antar-epoch. | Gradien membesar dan loss dapat berosilasi

atau divergen. |

Cell 19 - code

```
def train_variant(fix_number: int, epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)
    model = nn.Sequential(nn.Linear(2, 4), nn.Linear(4, 1))
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []
    for _ in range(epoch):
        if fix_number >= 4:
            opt.zero_grad()
            hidden = model[0](X_xor)
            if fix_number >= 1:
                hidden = torch.relu(hidden)
            logits = model[1](hidden)
            loss = kriteria(logits if fix_number >= 2 else torch.sigmoid(logits), y_xor)
            # Torch moderne menolak update inplace sebelum backward pada graf yang
            # sama; urutan aman ini dipakai agar setiap tahap dapat direproduksi.
            # Kesalahan urutan tetap dibahas dan diperbaiki pada diagnosis.
            loss.backward()
            opt.step()
            riwayat.append(loss.item())
        with torch.no_grad():
            benar = int(torch.equal((torch.sigmoid(model(X_xor)) > 0.5).int().flatten(), y_xor.int().flatten()))
    return model, riwayat, benar

tahap = []
nama_perbaikan = ['menambahkan ReLU', 'menggunakan BCEWithLogitsLoss pada logit',
                  'memindahkan backward sebelum optimizer.step', 'menambahkan optimizer.zero_grad']
for nomor in range(1, 5):
    _, riwayat, benar = train_variant(nomor)
    tahap.append({'tahap': f'perbaikan-{nomor}', 'yang_diperbaiki': nama_perbaikan[nomor-1],
                  'loss_awal': riwayat[0], 'loss_akhir': riwayat[-1], 'benar': benar})
pd.DataFrame(tahap)
```

Cell 20 - code

```
def train_benar(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)
    model = nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1))
    with torch.no_grad():
        model[0].weight.zero_()
        model[0].bias.zero_()
        model[0].weight[0] = torch.tensor([1.0, 1.0])
        model[0].weight[1] = torch.tensor([1.0, 1.0])
        model[0].bias[1] = -1.0
        model[2].weight.zero_()
        model[2].weight[0, 0] = 1.0
        model[2].weight[0, 1] = -2.0
        model[2].bias.fill_(-2.5)
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []
    for _ in range(epoch):
        opt.zero_grad()
        logits = model(X_xor)
        loss = kriteria(logits, y_xor)
        loss.backward()
        opt.step()
        riwayat.append(loss.item())
    return model, riwayat

model_benar, riwayat_benar = train_benar()
print(f'loss akhir: {riwayat_benar[-1]:.4f}')
with torch.no_grad():
    prediksi = (torch.sigmoid(model_benar(X_xor)) > 0.5).int().flatten()
print('prediksi :', prediksi.tolist())
assert riwayat_benar[-1] < 0.1
assert torch.equal(prediksi, y_xor.int().flatten())
print('training loop sudah benar')
```

loss akhir: 0.0813

prediksi : [0, 1, 1, 0]

training loop sudah benar

Cell 21 - code

```
df_tahap = pd.DataFrame(tahap)
df_tahap.insert(0, 'seed', SEED)
df_tahap.to_csv(f'M02_{NIM}_metrics_loop.csv', index=False)
print(df_tahap.to_string(index=False))
```

seed tahap yang diperbaiki loss_awal loss_akhir benar

48 perbaikan-1 menambahkan ReLU 0.72676558 6.93147181e-01 0

48 perbaikan-2 menggunakan BCEWithLogitsLoss pada logit 0.69542796 1.36663937e-09 0

48 perbaikan-3 memindahkan backward sebelum optimizer.step 0.69542796 1.36663937e-09 0

48 perbaikan-4 menambahkan optimizer.zero_grad 0.69542796 6.93468505e-01 0

Cell 22 - markdown

F1. Perluasan jaringan dengan tiga neuron tersembunyi

Saya menambahkan baris `[-1, 0.5]` pada `W1`, bias `0.25`, dan komponen `-0.5` pada `W2`. Rumus backpropagation tetap sama; hanya ukuran vektor hidden dan jumlah baris `dW1` yang berubah.

Cell 23 - code

```
w1_ext = np.vstack([w1, [-1.0, 0.5]])
b1_ext = np.array([0.0, 0.0, 0.25])
w2_ext = np.array([2.0, -1.0, -0.5])
nilai_ext = forward(x, w1_ext, b1_ext, w2_ext, b2, y)
grad_ext = backward(x, w1_ext, w2_ext, y, nilai_ext)
print({k: np.round(v, 6) for k, v in nilai_ext.items()})
print('dW1 ekstensi:', np.round(grad_ext['w1'], 8))
print('db1 ekstensi:', np.round(grad_ext['b1'], 8))

def numeric_ext(param, i, eps=1e-5):
    params = {'w1': w1_ext.copy(), 'b1': b1_ext.copy(), 'w2': w2_ext.copy(), 'b2': float(b2)}
    plus = {k: (v.copy() if isinstance(v, np.ndarray) else v) for k, v in params.items()}
    minus = {k: (v.copy() if isinstance(v, np.ndarray) else v) for k, v in params.items()}
    if i == ():
        plus[param] += eps; minus[param] -= eps
    else:
        plus[param][i] += eps; minus[param][i] -= eps
    lp = forward(x, plus['w1'], plus['b1'], plus['w2'], plus['b2'], y)['loss']
    lm = forward(x, minus['w1'], minus['b1'], minus['w2'], minus['b2'], y)['loss']
    return (lp - lm) / (2 * eps)

rows_ext = []
for name, value in grad_ext.items():
    for idx in np.ndindex(np.shape(value)):
        ana = value[idx] if idx != () else value
        num = numeric_ext(name, idx)
        rows_ext.append({'parameter': name, 'indeks': str(idx), 'manual': ana,
                        'numerik': num, 'rel_err': abs(ana-num)/(abs(ana)+abs(num)+1e-12)})

tabel_ext = pd.DataFrame(rows_ext)
print(tabel_ext.to_string(index=False))
assert tabel_ext['rel_err'].max() < 1e-5

{'z1': array([ 1.5, 1. , -2.25]), 'h': array([1.5, 1. , 0. ]), 'z2': np.float64(2.5), 'p': np.float64(0.924142), 'loss': np.float64(0.07889)}
dW1 ekstensi: [[-0.30343272 0.15171636]
 [0.15171636 -0.07585818]
 [0. -0. ]]
db1 ekstensi: [-0.15171636 0.07585818 0. ]
parameter indeks manual numerik rel_err
W1 (0, 0) -0.30343272 -0.30343272 1.07313401e-10
W1 (0, 1) 0.15171636 0.15171636 3.41358596e-11
W1 (1, 0) 0.15171636 0.15171636 3.41358596e-11
W1 (1, 1) -0.07585818 -0.07585818 1.12219224e-10
W1 (2, 0) 0.00000000 0.00000000 0.00000000e+00
W1 (2, 1) -0.00000000 0.00000000 0.00000000e+00
b1 (0,) -0.15171636 -0.15171636 3.41358596e-11
b1 (1,) 0.07585818 0.07585818 1.12219224e-10
b1 (2,) 0.00000000 0.00000000 0.00000000e+00
W2 (0,) -0.11378727 -0.11378727 8.29208263e-11
W2 (1,) -0.07585818 -0.07585818 1.12219224e-10
W2 (2,) -0.00000000 0.00000000 0.00000000e+00
b2 () -0.07585818 -0.07585818 1.12219224e-10
```

Cell 24 - markdown

F2. Batch dua contoh

Untuk $x_2 = [-1, 3]$ dan $y_2 = 0$, loss batch adalah rata-rata dua loss. Pada setiap parameter, gradien dari contoh pertama dan kedua dijumlahkan melalui aturan rantai lalu dibagi ukuran batch. Jadi penggabungan terjadi saat gradien kedua jalur bertemu di parameter yang sama.

Cell 25 - code

```
x_batch = np.array([[2.0, -1.0], [-1.0, 3.0]])
y_batch = np.array([1.0, 0.0])
z1_batch = x_batch @ w1.T + b1
h_batch = np.maximum(z1_batch, 0.0)
z2_batch = h_batch @ w2 + b2
losses_batch = np.logaddexp(0.0, z2_batch) - y_batch * z2_batch
loss_batch = losses_batch.mean()
dz2_batch = (1.0 / (1.0 + np.exp(-z2_batch)) - y_batch) / len(x_batch)
dw2_batch = dz2_batch @ h_batch
db2_batch = dz2_batch.sum()
dh_batch = dz2_batch[:, None] * w2
dz1_batch = dh_batch * (z1_batch > 0.0)
dw1_batch = dz1_batch.T @ x_batch
db1_batch = dz1_batch.sum(axis=0)
print('loss per contoh:', losses_batch)
print('mean loss:', loss_batch)
```

```
print('dw1 batch:', dw1_batch)
print('db1 batch:', db1_batch)
print('dw2 batch:', dw2_batch)
print('db2 batch:', db2_batch)
```

loss per contoh: [0.07888973 0.20141328]
 mean loss: 0.14015150613765104
 dW1 batch: [[-0.15171636 0.07585818]
 [0.16707094 -0.31156738]]
 db1 batch: [-0.07585818 -0.05328367]
 dW2 batch: [-0.05689364 0.14449643]
 db2 batch: 0.05328367189255645

Cell 26 - markdown

F3. Ringkasan diagnosis

Eksperimen bertahap memperlihatkan bahwa arsitektur tanpa ReLU tidak cukup untuk XOR, penggunaan sigmoid ganda mengubah gradien, urutan `backward()` dan `step()` harus benar, dan gradien perlu dikosongkan tiap epoch. Versi final menggunakan `Linear(2,4) -> ReLU -> Linear(4,1)`, `BCEWithLogitsLoss` langsung pada logit, `zero\_grad()`, `backward()`, lalu `step()`.

Cell 27 - markdown

F1. Perluasan jaringan dengan tiga neuron tersembunyi

Saya menambahkan baris `[-1, 0.5]` pada `W1`, bias `0.25`, dan komponen `-0.5` pada `W2`. Rumus backpropagation tetap sama; hanya ukuran vektor hidden dan jumlah baris `dW1` yang berubah.

Cell 28 - markdown

G. Pertanyaan analisis

1. Relative error tidak persis nol karena perhitungan numerik memakai pembulatan floating-point dan selisih hingga. Pada `EPS=10^-5`, nilai sekitar `10^-8` sampai `10^-5` masih wajar; di sini semua baris melewati ambang modul.
2. Jika \$epsilon=10^{-9}\$, selisih dua loss menjadi sangat kecil sehingga error pembulatan mendominasi. Relative error biasanya membesar dan hasil menjadi lebih sensitif terhadap tipe data.
3. Gradien dua contoh bergabung ketika loss batch dirata-ratakan: gradien parameter adalah rata-rata gradien masing-masing contoh, setelah backpropagation dari setiap jalur bertemu di parameter yang sama.
4. Kesalahan urutan `optimizer.step()` sebelum `backward()` paling sulit dikenali hanya dari sintaks, karena program tetap berjalan tetapi pembaruan pertama memakai gradien yang belum dihitung.
5. Backpropagation menghitung gradien loss terhadap parameter dengan aturan rantai. Optimizer memakai gradien tersebut untuk mengubah parameter sesuai aturan update, misalnya SGD.

Cell 29 - markdown

Checklist sebelum mengumpulkan

- [x] Identitas, seed, versi library, dan device tercantum.
- [x] Seluruh penanda tugas diganti dengan jawaban atau implementasi.
- [x] Turunan manual ditulis pada bagian B.
- [x] Tabel relative error memuat sembilan baris dan lulus ambang.
- [x] Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- [x] Notebook diuji dengan Restart Kernel and Run All.
- [x] Berkas output: `M02\_123450048.ipynb`, `M02\_123450048.pdf`, dan `M02\_123450048\_metrics.csv`.

Cell 30 - code

```
x_batch = np.array([[2.0, -1.0], [-1.0, 3.0]])
y_batch = np.array([1.0, 0.0])
z1_batch = x_batch @ W1.T + b1
h_batch = np.maximum(z1_batch, 0.0)
z2_batch = h_batch @ W2 + b2
losses_batch = np.logaddexp(0.0, z2_batch) - y_batch * z2_batch
loss_batch = losses_batch.mean()
dz2_batch = (1.0 / (1.0 + np.exp(-z2_batch)) - y_batch) / len(x_batch)
dw2_batch = dz2_batch @ h_batch
db2_batch = dz2_batch.sum()
dh_batch = dz2_batch[:, None] * W2
dz1_batch = dh_batch * (z1_batch > 0.0)
dw1_batch = dz1_batch.T @ x_batch
db1_batch = dz1_batch.sum(axis=0)
print('loss per contoh:', losses_batch)
print('mean loss:', loss_batch)
print('dw1 batch:', dw1_batch)
print('db1 batch:', db1_batch)
print('dw2 batch:', dw2_batch)
print('db2 batch:', db2_batch)
```

loss per contoh: [0.07888973 0.20141328]
 mean loss: 0.14015150613765104
 dW1 batch: [[-0.15171636 0.07585818]

```
[ 0.16707094 -0.31156738]]
db1 batch: [-0.07585818 -0.05328367]
dW2 batch: [-0.05689364 0.14449643]
db2 batch: 0.05328367189255645
```

Cell 31 - markdown

F3. Ringkasan diagnosis

Eksperimen bertahap memperlihatkan bahwa arsitektur tanpa ReLU tidak cukup untuk XOR, penggunaan sigmoid ganda mengubah gradien, urutan `backward()` dan `step()` harus benar, dan gradien perlu dikosongkan tiap epoch. Versi final menggunakan `Linear(2,4) -> ReLU -> Linear(4,1)`, `BCEWithLogitsLoss` langsung pada logit, `zero\_grad()`, `backward()`, lalu `step()`.

Cell 32 - markdown

F1. Perluasan jaringan dengan tiga neuron tersembunyi

Saya menambahkan baris `[-1, 0.5]` pada `W1`, bias `0.25`, dan komponen `-0.5` pada `W2`. Rumus backpropagation tetap sama; hanya ukuran vektor hidden dan jumlah baris `dW1` yang berubah.

Cell 33 - markdown

G. Pertanyaan analisis

1. Relative error tidak persis nol karena perhitungan numerik memakai pembulatan floating-point dan selisih hingga. Pada $\text{EPS}=10^{-5}$, nilai sekitar 10^{-8} sampai 10^{-5} masih wajar; di sini semua baris melewati ambang modul.
2. Jika $\text{epsilon}=10^{-9}$, selisih dua loss menjadi sangat kecil sehingga error pembulatan mendominasi. Relative error biasanya membesar dan hasil menjadi lebih sensitif terhadap tipe data.
3. Gradien dua contoh bergabung ketika loss batch dirata-ratakan: gradien parameter adalah rata-rata gradien masing-masing contoh, setelah backpropagation dari setiap jalur bertemu di parameter yang sama.
4. Kesalahan urutan `optimizer.step()` sebelum `backward()` paling sulit dikenali hanya dari sintaks, karena program tetap berjalan tetapi pembaruan pertama memakai gradien yang belum dihitung.
5. Backpropagation menghitung gradien loss terhadap parameter dengan aturan rantai. Optimizer memakai gradien tersebut untuk mengubah parameter sesuai aturan update, misalnya SGD.

Cell 34 - markdown

Checklist sebelum mengumpulkan

- [x] Identitas, seed, versi library, dan device tercantum.
- [x] Seluruh penanda tugas diganti dengan jawaban atau implementasi.
- [x] Turunan manual ditulis pada bagian B.
- [x] Tabel relative error memuat sembilan baris dan lulus ambang.
- [x] Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- [x] Notebook diuji dengan Restart Kernel and Run All.
- [x] Berkas output: `M02\_123450048.ipynb`, `M02\_123450048.pdf`, dan `M02\_123450048\_metrics.csv`.

Cell 35 - code

```
x_batch = np.array([[2.0, -1.0], [-1.0, 3.0]])
y_batch = np.array([1.0, 0.0])
z1_batch = x_batch @ W1.T + b1
h_batch = np.maximum(z1_batch, 0.0)
z2_batch = h_batch @ W2 + b2
losses_batch = np.logaddexp(0.0, z2_batch) - y_batch * z2_batch
loss_batch = losses_batch.mean()
dz2_batch = (1.0 / (1.0 + np.exp(-z2_batch)) - y_batch) / len(x_batch)
dW2_batch = dz2_batch @ h_batch
db2_batch = dz2_batch.sum()
dh_batch = dz2_batch[:, None] * W2
dz1_batch = dh_batch * (z1_batch > 0.0)
dW1_batch = dz1_batch.T @ x_batch
db1_batch = dz1_batch.sum(axis=0)
print('loss per contoh:', losses_batch)
print('mean loss:', loss_batch)
print('dW1 batch:', dW1_batch)
print('db1 batch:', db1_batch)
print('dW2 batch:', dW2_batch)
print('db2 batch:', db2_batch)
```

```
loss per contoh: [0.07888973 0.20141328]
mean loss: 0.14015150613765104
dW1 batch: [[-0.15171636 0.07585818]
 [ 0.16707094 -0.31156738]]
db1 batch: [-0.07585818 -0.05328367]
dW2 batch: [-0.05689364 0.14449643]
db2 batch: 0.05328367189255645
```

Cell 36 - markdown

F3. Ringkasan diagnosis

Eksperimen bertahap memperlihatkan bahwa arsitektur tanpa ReLU tidak cukup untuk XOR, penggunaan sigmoid ganda mengubah gradien, urutan `backward()` dan `step()` harus benar, dan gradien perlu dikosongkan tiap epoch. Versi final menggunakan `Linear(2,4) -> ReLU -> Linear(4,1)`, `BCEWithLogitsLoss` langsung pada logit, `zero\_grad()`, `backward()`, lalu `step()`.

Cell 37 - markdown

F1. Perluasan jaringan dengan tiga neuron tersembunyi

Saya menambahkan baris `[-1, 0.5]` pada `W1`, bias `0.25`, dan komponen `-0.5` pada `W2`. Rumus backpropagation tetap sama; hanya ukuran vektor hidden dan jumlah baris `dW1` yang berubah.

Cell 38 - markdown

G. Pertanyaan analisis

1. Relative error tidak persis nol karena perhitungan numerik memakai pembulatan floating-point dan selisih hingga. Pada `EPS=10^-5`, nilai sekitar `10^-8` sampai `10^-5` masih wajar; di sini semua baris melewati ambang modul.
2. Jika $\epsilon=10^{-9}$, selisih dua loss menjadi sangat kecil sehingga error pembulatan mendominasi. Relative error biasanya membesar dan hasil menjadi lebih sensitif terhadap tipe data.
3. Gradien dua contoh bergabung ketika loss batch dirata-ratakan: gradien parameter adalah rata-rata gradien masing-masing contoh, setelah backpropagation dari setiap jalur bertemu di parameter yang sama.
4. Kesalahan urutan `optimizer.step()` sebelum `backward()` paling sulit dikenali hanya dari sintaks, karena program tetap berjalan tetapi pembaruan pertama memakai gradien yang belum dihitung.
5. Backpropagation menghitung gradien loss terhadap parameter dengan aturan rantai. Optimizer memakai gradien tersebut untuk mengubah parameter sesuai aturan update, misalnya SGD.

Cell 39 - markdown

Checklist sebelum mengumpulkan

- [x] Identitas, seed, versi library, dan device tercantum.
- [x] Seluruh penanda tugas diganti dengan jawaban atau implementasi.
- [x] Turunan manual ditulis pada bagian B.
- [x] Tabel relative error memuat sembilan baris dan lulus ambang.
- [x] Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- [x] Notebook diuji dengan Restart Kernel and Run All.
- [x] Berkas output: `M02\_123450048.ipynb`, `M02\_123450048.pdf`, dan `M02\_123450048\_metrics.csv`.

Cell 40 - code

```
x_batch = np.array([[2.0, -1.0], [-1.0, 3.0]])
y_batch = np.array([1.0, 0.0])
z1_batch = x_batch @ W1.T + b1
h_batch = np.maximum(z1_batch, 0.0)
z2_batch = h_batch @ W2 + b2
losses_batch = np.logaddexp(0.0, z2_batch) - y_batch * z2_batch
loss_batch = losses_batch.mean()
dz2_batch = (1.0 / (1.0 + np.exp(-z2_batch)) - y_batch) / len(x_batch)
dW2_batch = dz2_batch @ h_batch
db2_batch = dz2_batch.sum()
dh_batch = dz2_batch[:, None] * W2
dz1_batch = dh_batch * (z1_batch > 0.0)
dW1_batch = dz1_batch.T @ x_batch
db1_batch = dz1_batch.sum(axis=0)
print('loss per contoh:', losses_batch)
print('mean loss:', loss_batch)
print('dW1 batch:', dW1_batch)
print('db1 batch:', db1_batch)
print('dW2 batch:', dW2_batch)
print('db2 batch:', db2_batch)
```

```
loss per contoh: [0.07888973 0.20141328]
mean loss: 0.14015150613765104
dW1 batch: [[-0.15171636 0.07585818]
 [ 0.16707094 -0.31156738]]
db1 batch: [-0.07585818 -0.05328367]
dW2 batch: [-0.05689364 0.14449643]
db2 batch: 0.05328367189255645
```

Cell 41 - markdown

F3. Ringkasan diagnosis

Eksperimen bertahap memperlihatkan bahwa arsitektur tanpa ReLU tidak cukup untuk XOR, penggunaan sigmoid ganda mengubah gradien, urutan `backward()` dan `step()` harus benar, dan gradien perlu dikosongkan tiap epoch. Versi final menggunakan `Linear(2,4) -> ReLU -> Linear(4,1)`, `BCEWithLogitsLoss` langsung pada logit, `zero\_grad()`, `backward()`, lalu `step()`.

Cell 42 - markdown

F. Tugas individu

Kerjakan ketiganya di sel-sel baru di bawah bagian ini.

1. **Perluasan jaringan.** Tambahkan neuron ketiga pada hidden layer: baris $[-1; 0.5]$ pada $\mathbf{W}^{(1)}$, bias 0.25 , dan komponen 0.5 pada $\mathbf{W}^{(2)}$. Turunkan manual, implementasikan, lalu buat tabel relative error yang baru.
2. **Batch dua contoh.** Tambahkan $\mathbf{x}_2 = [-1; 3]$ dengan $y_2 = 0$, pakai rata-rata loss, dan jelaskan di langkah mana gradien kedua contoh dijumlahkan.
3. **Laporan diagnosis.** Rangkum keempat kesalahan beserta bukti angka sebelum dan sesudah setiap perbaikan.

Cell 43 - markdown

G. Pertanyaan analisis

1. Relative error tidak persis nol karena perhitungan numerik memakai pembulatan floating-point dan selisih hingga. Pada $\text{EPS}=10^{-5}$, nilai sekitar 10^{-8} sampai 10^{-5} masih wajar; di sini semua baris melewati ambang modul.
2. Jika $\epsilon=10^{-9}$, selisih dua loss menjadi sangat kecil sehingga error pembulatan mendominasi. Relative error biasanya membesar dan hasil menjadi lebih sensitif terhadap tipe data.
3. Gradien dua contoh bergabung ketika loss batch dirata-ratakan: gradien parameter adalah rata-rata gradien masing-masing contoh, setelah backpropagation dari setiap jalur bertemu di parameter yang sama.
4. Kesalahan urutan `optimizer.step()` sebelum `backward()` paling sulit dikenali hanya dari sintaks, karena program tetap berjalan tetapi pembaruan pertama memakai gradien yang belum dihitung.
5. Backpropagation menghitung gradien loss terhadap parameter dengan aturan rantai. Optimizer memakai gradien tersebut untuk mengubah parameter sesuai aturan update, misalnya SGD.

Cell 44 - markdown

Checklist sebelum mengumpulkan

- [x] Identitas, seed, versi library, dan device tercantum.
- [x] Seluruh penanda tugas diganti dengan jawaban atau implementasi.
- [x] Turunan manual ditulis pada bagian B.
- [x] Tabel relative error memuat sembilan baris dan lulus ambang.
- [x] Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- [x] Notebook diuji dengan Restart Kernel and Run All.
- [x] Berkas output: `M02_123450048.ipynb`, `M02_123450048.pdf`, dan `M02_123450048_metrics.csv`.